

UNIVERSITÀ
DEGLI STUDI
FIRENZE

Università degli Studi di Firenze

Dipartimento di Ingegneria dell'Informazione

MBPConfigurator

Configuratore di PC

Autori:

Tommaso Lapenna

Lucas Melo Alves

Samuele Baldi

N° matricola:

7112937

7777777

7116707

Corso:

Ingegneria del Software

Docente del corso:

Prof. Enrico Vicario

Anno Accademico 2025–2026

Indice

1 Introduzione **2**
1.1 Statement . . . 2
1.2 Obiettivo e Requisiti . . . 2
1.3 Tecnologie e Strumenti Utilizzati . . . 2

2 Progettazione **3**
2.1 Architettura e Design . . . 3
2.2 Use Case Diagram . . . 4
2.3 Use Case Template . . . 4
2.4 Mock-Ups . . . 13
2.5 Navigation Diagram . . . 16
2.6 Class Diagram . . . 16
2.7 ER Diagram e Modello Relazionale . . . 16

3 Implementazione **17**
3.1 Domain . . . 17
3.1.1 Model . . . 17
3.1.2 Event . . . 21
3.1.3 Rule . . . 22
3.2 Application . . . 24
3.3 ORM . . . 24

4 Design Pattern e Strutture Dati **25**
4.1 Specification . . . 25
4.1.1 Logica . . . 27
4.2 Observer . . . 27
4.3 Memento . . . 27
4.4 Facade . . . 27
4.5 Altri Design Pattern . . . 27
4.6 Strutture Dati Utilizzate . . . 27

1 Introduzione

1.1 Statement

Il progetto **MPBConfigurator** modella un'applicazione che permette all'utente di creare una pc build, confrontare i componenti e verificarne la compatibilità ed il prezzo totale.

L'interfaccia del configuratore permette di inserire via via i componenti scelti, annullare le scelte effettuate o ripristinare l'annullamento. Inoltre sono visibili informazioni su cosa modificare e/o inserire per ottenere un pc funzionante tramite il sistema di compatibilità in tempo reale.

Tutto il catalogo dei componenti è disponibile all'utente tramite l'apposito database, quest'ultimo è gestito dall'operatore del sistema.

1.2 Obiettivo e Requisiti

???? maybe?

1.3 Tecnologie e Strumenti Utilizzati

Java 25	Implementazione dell'applicazione
IntelliJ	IDE
JUnit 5	Testing dell'applicazione
SQLite	Database dell'applicazione
PlantUML	Strumento di creazione dei diagrammi UML
Github	Versionamento e condivisione del codice

Tabella 1: Tecnologie utilizzate

Sono inoltre stati utilizzati i seguenti modelli per aiutare lo sviluppo del codice e la progettazione:

- Claude Sonnet 5.5
- Claude Agent
- Gemini Pro 3.1, Flash 3.5
- LLM locali: (Gemma 4 e4b, Qwen 3.5 9b, Nemotron 3-nano 4b)
- **GUI** rappresenta l'UI layer.
- **Application** rappresenta l'Application Layer, e si occuppa di gestire le richieste dell'utente e la coordinazioni tra i vari elementi (in modo da "risolvere" gli use cases).

- **Domain** rappresenta il Domain Layer: contiene il modello di tutte le entità del dominio in questione (tutti i tipi di componenti e le build), insieme a tutta la logica e regole che le gestiscono.
- **ORM** rappresenta il Persisistence Layer, definisce le repository interfaces deinite dal dominio.

2 Progettazione

2.1 Architettura e Design

L'applicazione è strutturata secondo un'architettura a livelli, inspirata al Domain Driven Desgin

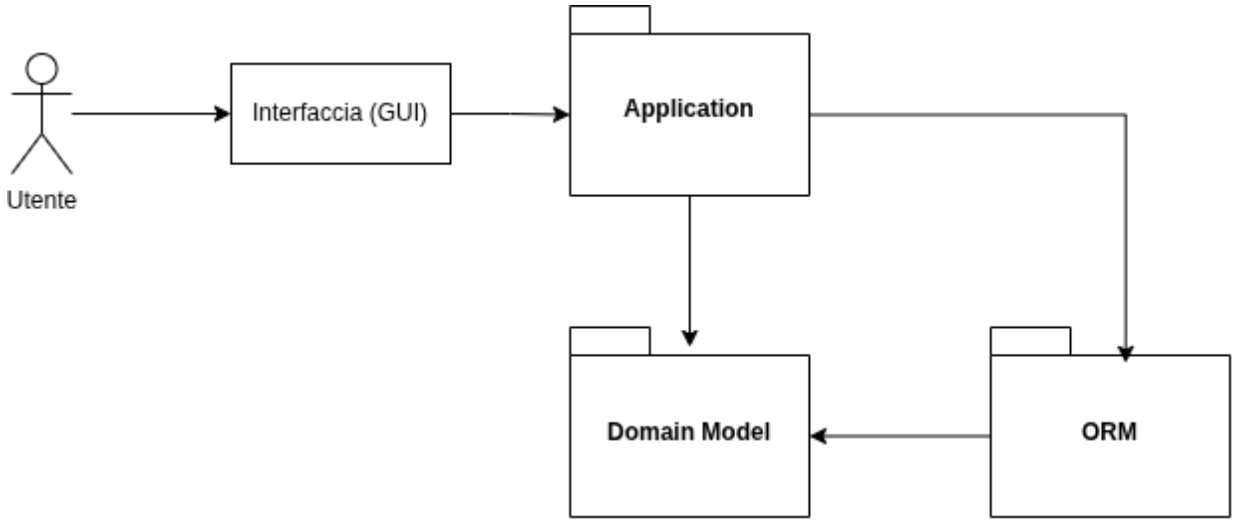


Figura 1: Package Diagram

NEL DIAGRAMMA MANCA IL DB
DEFINIZIONE LAYER

2.2 Use Case Diagram

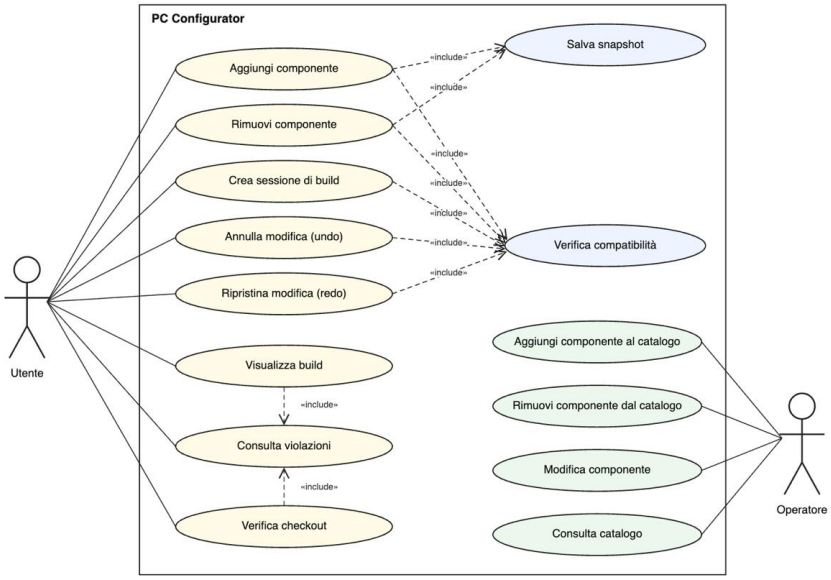


Figura 2: Use Case Diagram

2.3 Use Case Template

UC1 — Crea sessione di build	
Campo	**Contenuto**
Descrizione	L'utente avvia una nuova sessione di configurazione PC. Il sistema crea una `PcBuildSession` vuota con `PcBuilder` e `SnapshotHistory` inizializzati.
Livello	User goal
Attori	Utente
Pre-condizioni	L'utente è autenticato nel sistema.
Flusso base	1. L'utente seleziona «Nuova build». 2. Il sistema crea una `PcBuildSession` con `PcBuilder` vuoto. 3. Il sistema inizializza la `SnapshotHistory`. 4. Il sistema mostra l'interfaccia di configurazione con tutti gli slot vuoti.
Flussi alternativi	1a. L'utente sceglie di caricare una build salvata: il sistema ripristina la sessione precedente (→ UC6).
Post-condizioni	Una sessione di build attiva è disponibile.
Test	

UC2 — Aggiungi componente	
Campo	**Contenuto**
Descrizione	L'utente aggiunge un componente dal catalogo alla build corrente. Il sistema salva uno snapshot e verifica la compatibilità automaticamente.
Livello	User goal
Attori	Utente
Pre-condizioni	Una sessione di build è attiva. Il catalogo contiene almeno un componente.
Flusso base	1. L'utente esplora il catalogo e seleziona un componente. 2. L'utente clicca «Aggiungi alla build». 3. Il sistema aggiunge il componente tramite `PcBuildSession.add()`. 4. Il sistema salva uno snapshot nella `SnapshotHistory` «include UC-Snapshot». 5. Il sistema esegue la verifica di compatibilità «include UC-Verifica». 6. Il sistema aggiorna la vista della build e mostra eventuali violazioni.
Flussi alternativi	3a. La cardinalità massima per quel tipo è già raggiunta: il sistema mostra un errore e non aggiunge il componente. 5a. La verifica rileva violazioni: il sistema le elenca senza bloccare l'aggiunta.
Post-condizioni	Il componente è nella build. Uno snapshot è salvato. Le violazioni sono aggiornate.
Test	

UC3 — Rimuovi componente	
Campo	**Contenuto**
Descrizione	L'utente rimuove un componente dalla build corrente. Il sistema salva uno snapshot e aggiorna la verifica di compatibilità.
Livello	User goal
Attori	Utente
Pre-condizioni	Una sessione di build è attiva. La build contiene almeno un componente.
Flusso base	1. L'utente seleziona un componente nella build corrente. 2. L'utente clicca «Rimuovi». 3. Il sistema rimuove il componente tramite `PcBuildSession.remove()`. 4. Il sistema salva uno snapshot «include UC-Snapshot». 5. Il sistema esegue la verifica di compatibilità «include UC-Verifica». 6. Il sistema aggiorna la vista della build.
Flussi alternativi	3a. Il componente non è presente nella build: il sistema mostra un errore.
Post-condizioni	Il componente non è più nella build. Snapshot salvato. Violazioni aggiornate.
Test	

UC4 — Annulla modifica (undo)	
Campo	**Contenuto**
Descrizione	L'utente annulla l'ultima modifica apportata alla build, ripristinando lo stato precedente tramite `SnapshotHistory`.
Livello	User goal
Attori	Utente
Pre-condizioni	Una sessione di build è attiva. La `SnapshotHistory` contiene almeno uno snapshot precedente.
Flusso base	1. L'utente clicca «Annulla». 2. Il sistema ripristina lo stato precedente tramite `PcBuildSession.undo()`. 3. Il sistema esegue la verifica di compatibilità sullo stato ripristinato «include UC-Verifica». 4. Il sistema aggiorna la vista della build.
Flussi alternativi	2a. Non esistono snapshot precedenti: il sistema disabilita il pulsante «Annulla» e non esegue alcuna azione.
Post-condizioni	La build è ripristinata allo stato precedente. Le violazioni sono aggiornate.
Test	

UC5 — Ripristina modifica (redo)	
Campo	**Contenuto**
Descrizione	L'utente ripristina una modifica precedentemente annullata tramite la `SnapshotHistory`.
Livello	User goal
Attori	Utente
Pre-condizioni	Una sessione di build è attiva. È stata eseguita almeno un'operazione di undo.
Flusso base	1. L'utente clicca «Ripristina». 2. Il sistema recupera lo stato successivo tramite `PcBuildSession.redo()`. 3. Il sistema esegue la verifica di compatibilità «include UC-Verifica». 4. Il sistema aggiorna la vista della build.
Flussi alternativi	2a. Non esistono snapshot successivi: il sistema disabilita il pulsante «Ripristina».
Post-condizioni	La build è ripristinata allo stato successivo. Le violazioni sono aggiornate.
Test	

UC6 — Visualizza build	
Campo	**Contenuto**
Descrizione	L'utente visualizza la configurazione corrente della build con tutti i componenti selezionati, le violazioni di compatibilità e il wattaggio totale.
Livello	User goal
Attori	Utente
Pre-condizioni	Una sessione di build è attiva.
Flusso base	1. Il sistema mostra i componenti presenti tramite `BuildView` (`getBuild()`). 2. Il sistema mostra le violazioni di compatibilità «include UC7». 3. Il sistema mostra il wattaggio totale (`totalWattage()`).
Flussi alternativi	1a. La build è vuota: il sistema mostra gli slot vuoti per ogni `ComponentType`.
Post-condizioni	L'utente ha una vista aggiornata della build corrente.
Test	

UC7 — Consulta violazioni	
Campo	**Contenuto**
Descrizione	Il sistema mostra la lista delle regole di compatibilità violate nella build corrente, calcolate dal `CompatibilityChecker` tramite le 40 `BuildRule` di `DefaultRules`.
Livello	Subfunction
Attori	Utente (indiretto, tramite UC6)
Pre-condizioni	Una sessione di build è attiva. La verifica di compatibilità è stata eseguita.
Flusso base	1. Il sistema recupera le violazioni tramite `PcBuildSession.getViolations()`. 2. Il sistema mostra la lista con la descrizione di ogni regola violata. 3. Se non ci sono violazioni, mostra «Nessun problema rilevato».
Flussi alternativi	(nessuno)
Post-condizioni	L'utente è informato delle incompatibilità presenti nella build.
Test	

UC8 — Visualizza prezzo	
Campo	**Contenuto**
Descrizione	Il sistema mostra il prezzo di mercato suggerito della build corrente, calcolato come somma dei prezzi indicativi dei componenti. Il prezzo non è vincolante.
Livello	User goal
Attori	Utente
Pre-condizioni	Una sessione di build è attiva.
Flusso base	1. Il sistema somma i prezzi di tutti i componenti nella build. 2. Il sistema mostra il totale con la dicitura «Prezzo di mercato suggerito». 3. Il sistema mostra il disclaimer: «I prezzi sono indicativi e soggetti a variazione».
Flussi alternativi	1a. La build è vuota: il sistema mostra €0,00.
Post-condizioni	L'utente conosce la stima del prezzo della configurazione.
Test	

UC9 — Aggiungi al catalogo	
Campo	**Contenuto**
Descrizione	L'operatore aggiunge un nuovo componente al catalogo del sistema, specificandone tutte le caratteristiche tecniche.
Livello	User goal
Attori	Operatore
Pre-condizioni	L'operatore è autenticato. Il componente non è già presente nel catalogo.
Flusso base	1. L'operatore seleziona «Aggiungi componente» e sceglie il tipo. 2. Il sistema mostra il form con i campi richiesti (socket, form factor, ecc.). 3. L'operatore compila i campi e conferma. 4. Il sistema valida i dati inseriti. 5. Il sistema aggiunge il componente al catalogo. 6. Il sistema mostra la conferma dell'operazione.
Flussi alternativi	4a. I dati non sono validi (campo obbligatorio mancante): il sistema segnala l'errore e non salva. 5a. Un componente identico esiste già: il sistema segnala il duplicato.
Post-condizioni	Il componente è disponibile nel catalogo per tutti gli utenti.
Test	

UC10 — Rimuovi dal catalogo	
Campo	**Contenuto**
Descrizione	L'operatore rimuove un componente esistente dal catalogo.
Livello	User goal
Attori	Operatore
Pre-condizioni	L'operatore è autenticato. Il componente esiste nel catalogo.
Flusso base	1. L'operatore cerca e seleziona il componente nel catalogo. 2. L'operatore seleziona «Rimuovi». 3. Il sistema chiede conferma. 4. L'operatore conferma. 5. Il sistema rimuove il componente dal catalogo.
Flussi alternativi	4a. L'operatore annulla: il sistema non esegue alcuna modifica.
Post-condizioni	Il componente non è più disponibile nel catalogo.
Test	

UC11 — Modifica componente nel catalogo	
Campo	**Contenuto**
Descrizione	L'operatore modifica le informazioni di un componente esistente nel catalogo.
Livello	User goal
Attori	Operatore
Pre-condizioni	L'operatore è autenticato. Il componente esiste nel catalogo.
Flusso base	1. L'operatore cerca e seleziona il componente. 2. Il sistema mostra il form con i dati attuali. 3. L'operatore modifica i campi desiderati e conferma. 4. Il sistema valida i nuovi dati. 5. Il sistema aggiorna il componente nel catalogo.
Flussi alternativi	4a. I dati modificati non sono validi: il sistema segnala l'errore e non salva.
Post-condizioni	Il componente nel catalogo riflette i nuovi dati.
Test	

UC12 — Consulta catalogo	
Campo	**Contenuto**
Descrizione	L'operatore visualizza la lista completa dei componenti nel catalogo, con possibilità di filtrare per tipo.
Livello	User goal
Attori	Operatore
Pre-condizioni	L'operatore è autenticato.
Flusso base	1. L'operatore accede alla sezione catalogo. 2. Il sistema mostra tutti i componenti suddivisi per tipo (CPU, GPU, RAM, ...). 3. L'operatore può filtrare per tipo, marca o prezzo.
Flussi alternativi	2a. Il catalogo è vuoto: il sistema mostra «Nessun componente presente».
Post-condizioni	L'operatore ha una vista aggiornata del catalogo.
Test	

2.4 Mock-Ups

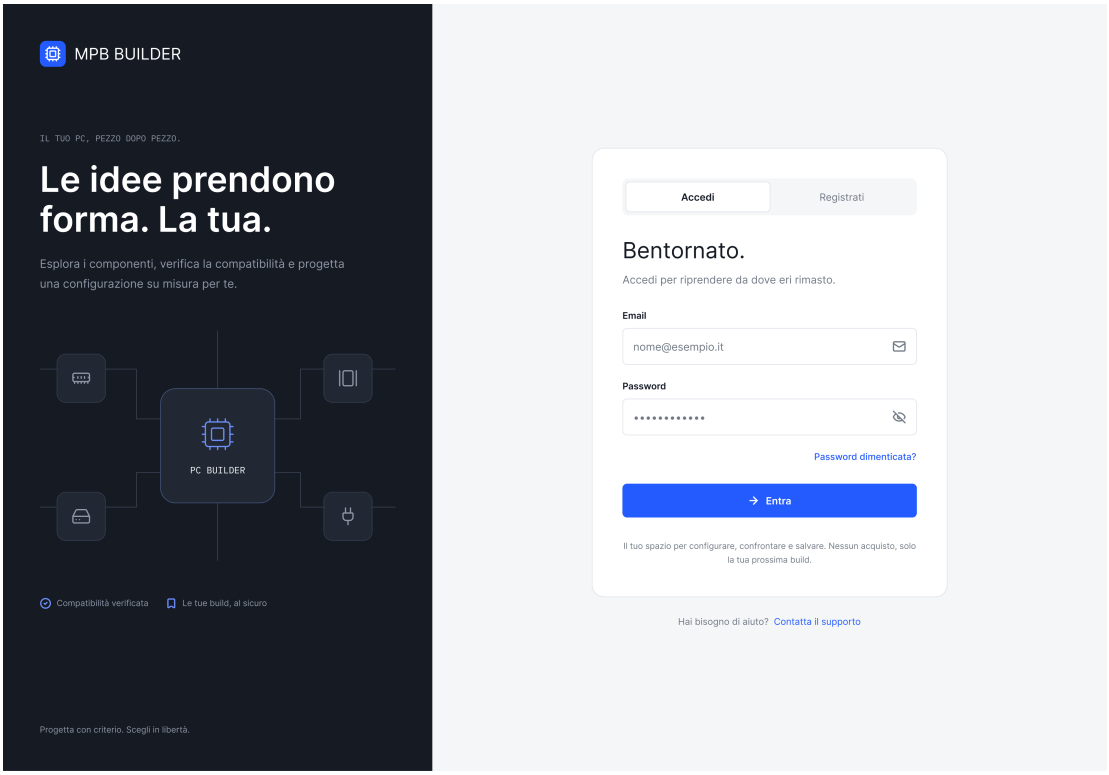


Figura 3: Mockup-1

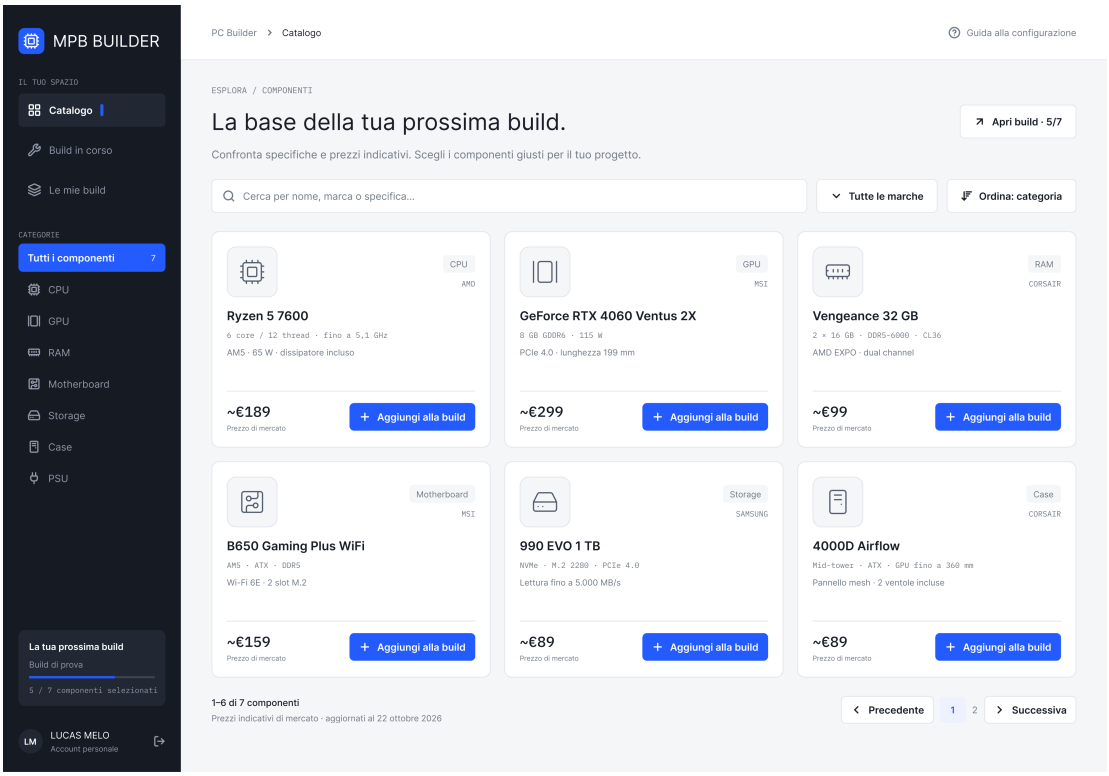


Figura 4: Mockup-2

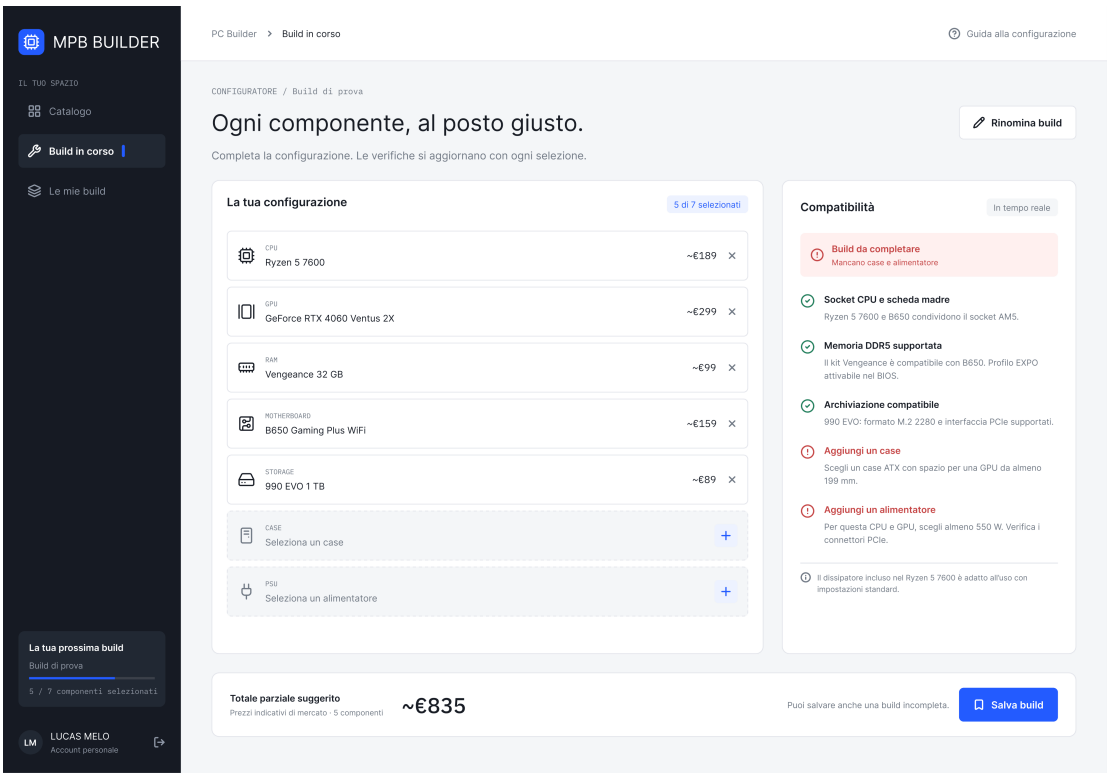


Figura 5: Mockup 3

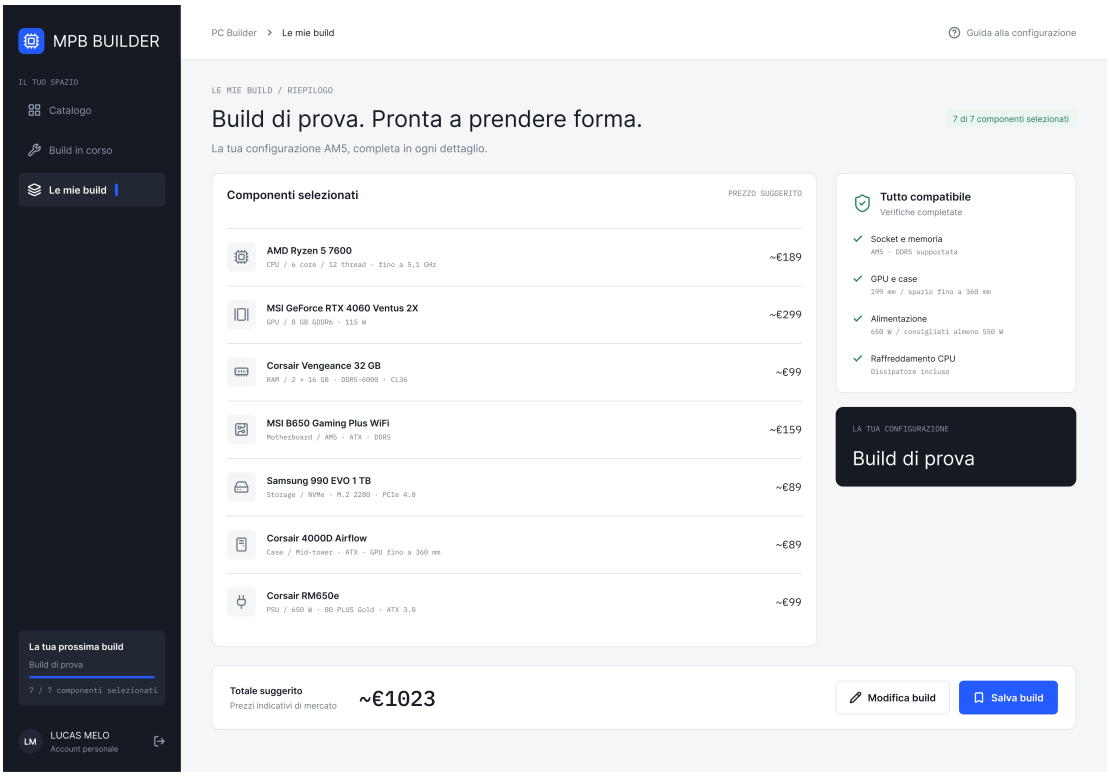


Figura 6: Mockup 4

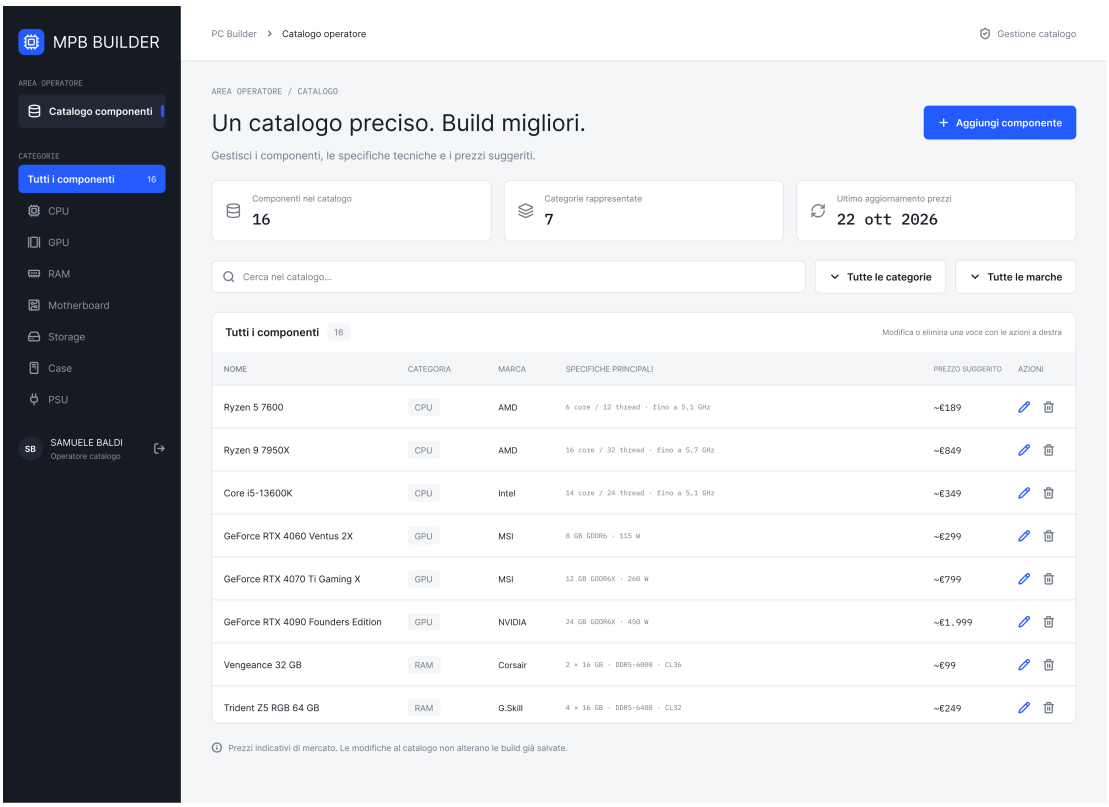


Figura 7: Mockup 5

2.5 Navigation Diagram

da fare

2.6 Class Diagram

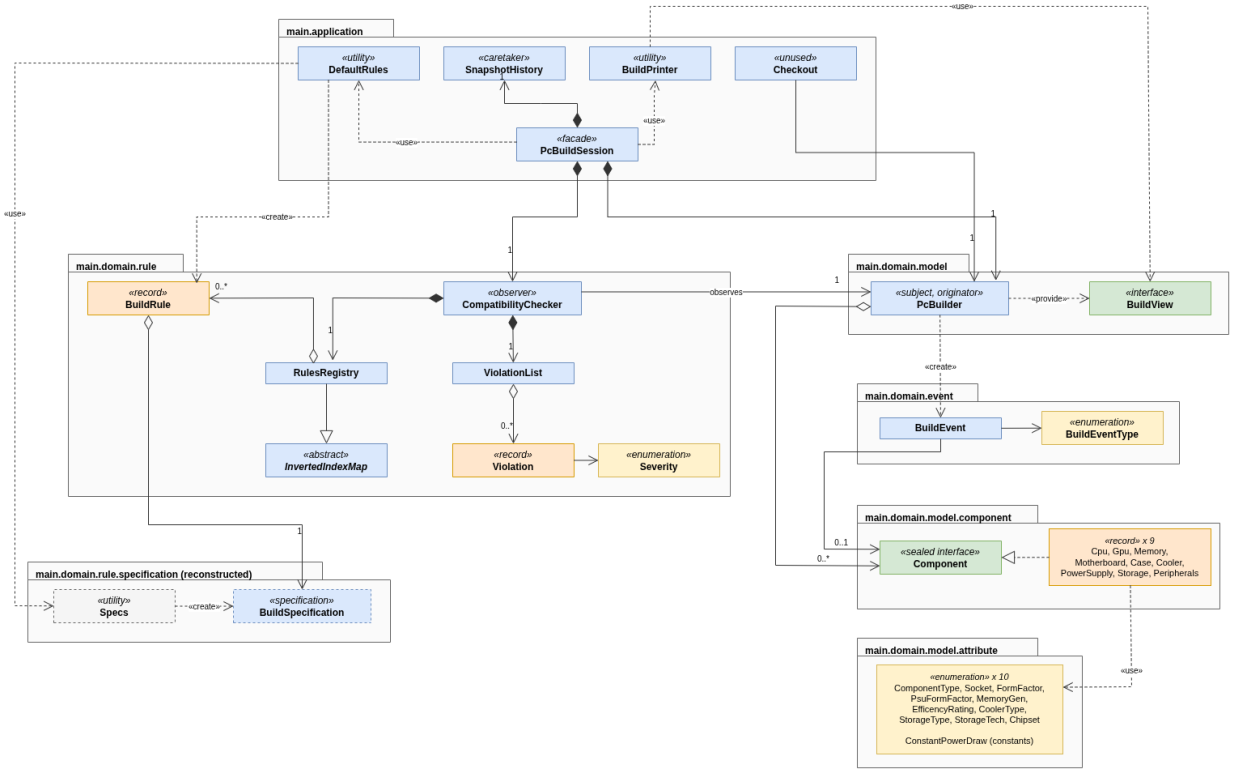


Figura 8: Enter Caption

2.7 ER Diagram e Modello Relazionale

deve farla il baldè

3 Implementazione

3.1 Domain

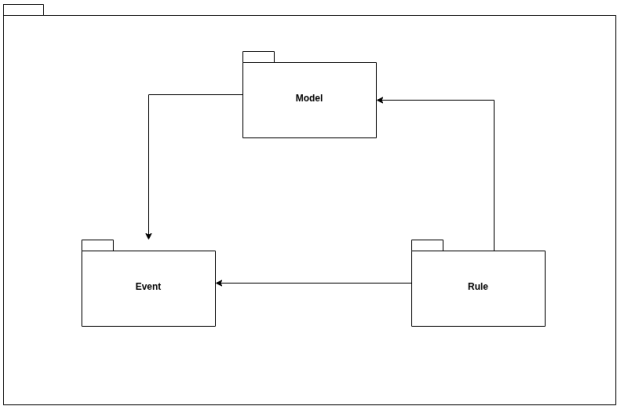


Figura 9: Domain Package Diagram

Il dominio contiene sia il modello che le sue business rule, nei corrispettivi pacchetti *.model* e *rule*

3.1.1 Model

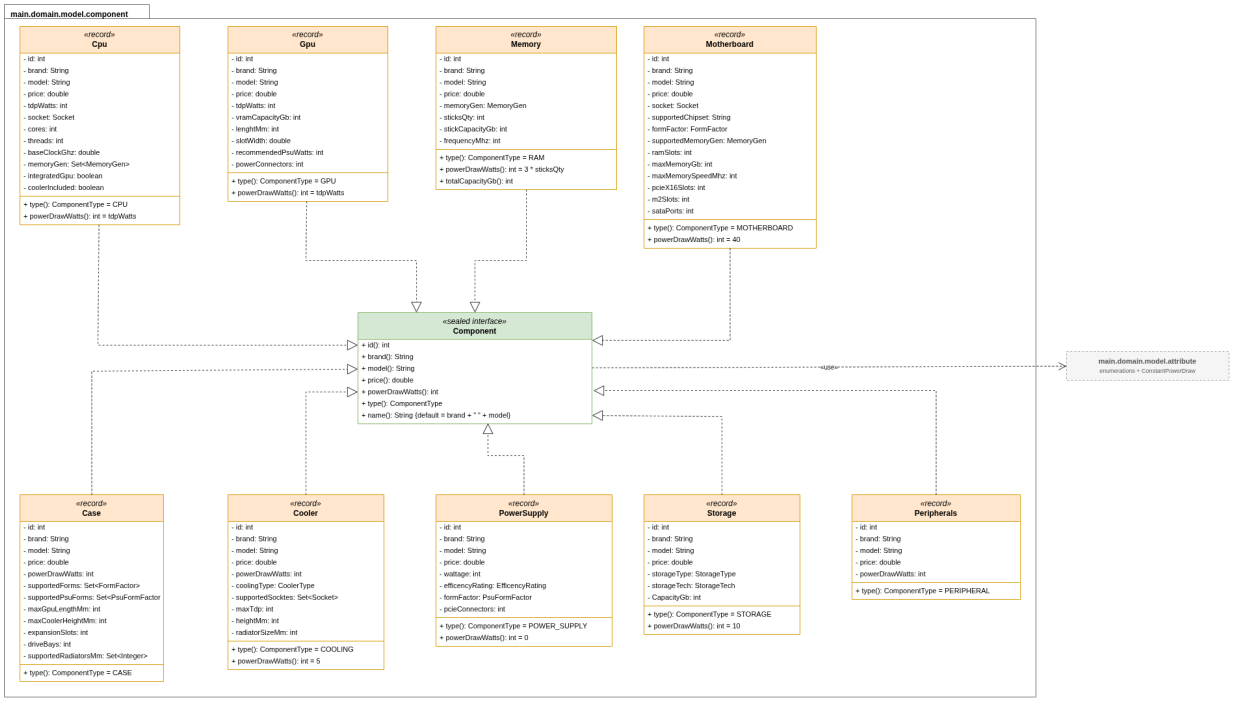


Figura 10: Enter Caption

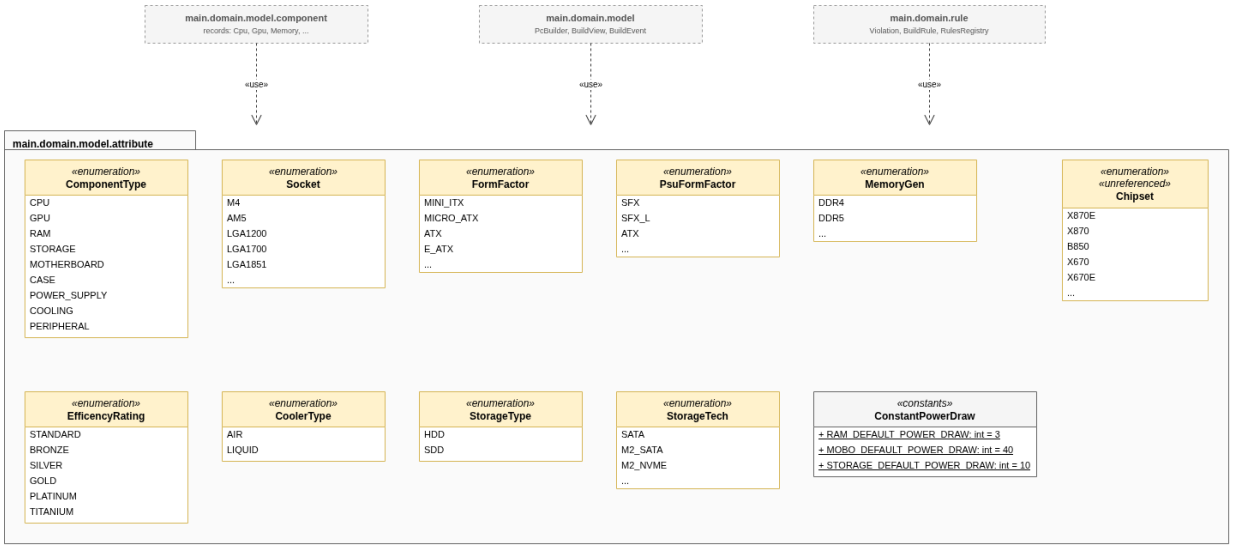


Figura 11: Enter Caption

Il modello è composto da due sottopacchetti:

- **component** contiene l'interfaccia chiusa *Component*, che viene implementata dalle tipologie possibili tipologie di componenti come *record*. La scelta di implementarli come record deriva dalla loro natura costante e immutabile della collezione di informazioni che rappresenta ogni componente, perciò è possibile sfruttare la sintassi più semplice e i vantaggi per la jvm.

```java
//Component.java
package main.domain.model.component;

import main.domain.model.attribute.ComponentType;

public sealed interface Component
        permits Case, Gpu, Peripherals, Cooler, Cpu,
        Memory, Motherboard, PowerSupply, Storage {

    int id();
    String brand();
    String model();
    double price();

    int powerDrawWatts();

    public ComponentType type();
    default String name() { return brand() + " " + model();
        }
```

```
19
20 }
```

Listing 1: A simple Java class

I record in questione sono i seguenti:

  - `Cpu`
  - `Gpu`
  - `Motherboard`
  - `Memory`
  - `Storage`
  - `Case`
  - `PowerSupply`
  - `Peripherals`

- **attribute** contiene invece le enumerazioni che determinano le tipologie le caratteristiche dei componenti. queste sono:
  - `ComponentType`
  - `Chipset`
  - `CoolerType`
  - `EfficencyRating`
  - `FormFactor`
  - `MemoryGen`
  - `PsuFormFactor`
  - `Socket`
  - `StorageTech`
  - `StorageType`
  - `ConstantPowerDraw` (non è un'enum a differenza degli altri)

```java
//ComponentType.java
package main.domain.model.attribute;

public enum ComponentType {
    CPU,
    GPU,
```

```
7       RAM,
8       STORAGE,
9       MOTHERBOARD,
10      CASE,
11      POWER_SUPPLY,
12      COOLING,
13      PERIPHERAL
14  }
```

Listing 2: A simple Java class

Tutti questi componenti verranno poi composti tra di loro per ottenere la build completa del pc, che rappresenta il punto centrale dell'applicazione. La classe che si occupa di questo è `PcBuilder`, che raccoglie tutti i componenti in un'*EnumMap*: le chiavi sono quelle fornite dall'enumerazione `ComponentType`, mentre i valori sono delle liste di elementi `Component`, che contengono i componenti per a seconda della corrispettiva tipologia.
Le liste possono contenere più elementi per tipologia, questo però è un comportamento consentito dal metodo `addComponent()` e da `CompatibilityChecker`: un utente può inserire ad esempio due cpu alla configurazione (magari per confrontarle in un attimo di decisione), questa irregolarità verrà poi segnalata in `ViolationList` e non sarà possibile effettuare il checkout finché queste persistono.

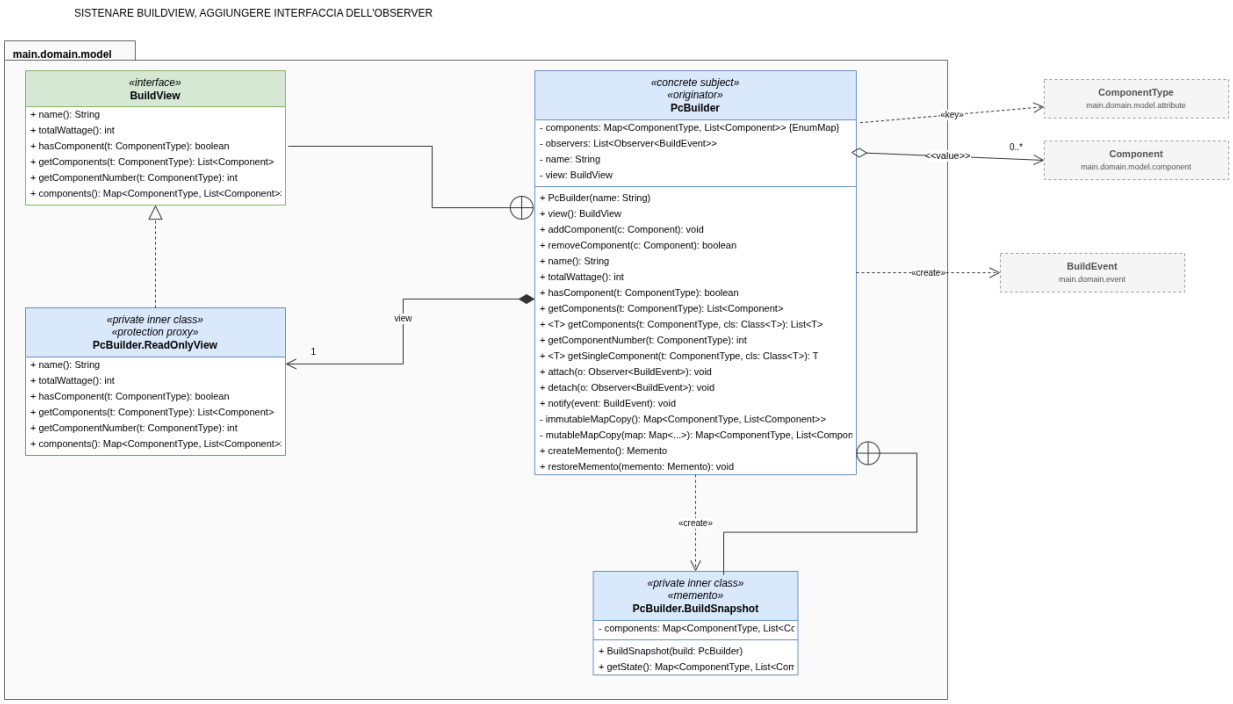


Figura 12: Enter Caption

`PcBuilder` contiene al suo interno due classi astratte:

- `ReadonlyView`: implementa l'interfaccia pubblica BuildView. Se questa interfaccia fosse implementata da PcBuilder direttamente, sarebbe possibile effettuare il cast `(PcBuilder) session.getBuild()...` e violare i limiti di sola lettura imposta dall'interfaccia. Implementare invece quetsa interfaccia in una classe privata che funge da *Proxy*, rende impossibile questo "escamotage".

- `BuildSnapshot`: permette l'implementazione corretta di memento. Questa classe privata è la migliore alternativa alla funzionalità *friend* del C++ (mancante in java) utilizzata in *GoF* per implementare memento: quindi BuildSnapshot implementa l'interfaccia pubblica `Memento` e il tipo che ritorna *createMemento* è il medesimo, così all'esterno non è possibile effettuare un cast alla classe privata i questione preservando la segretezza dello stato del memento al di fuori di *Originator*.

3.1.2 Event

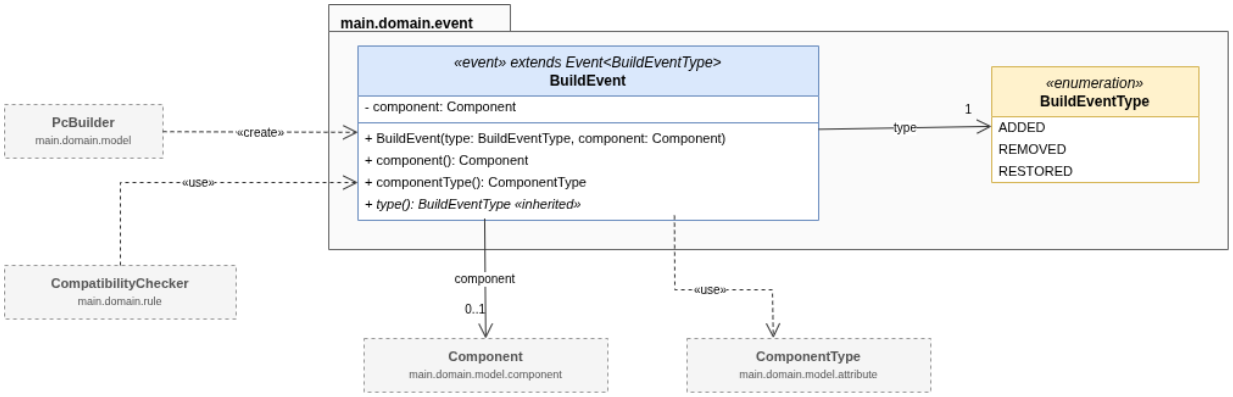


Figura 13: Enter Caption

Contiene le informazioni su che modifica è stata effettuata...

3.1.3 Rule

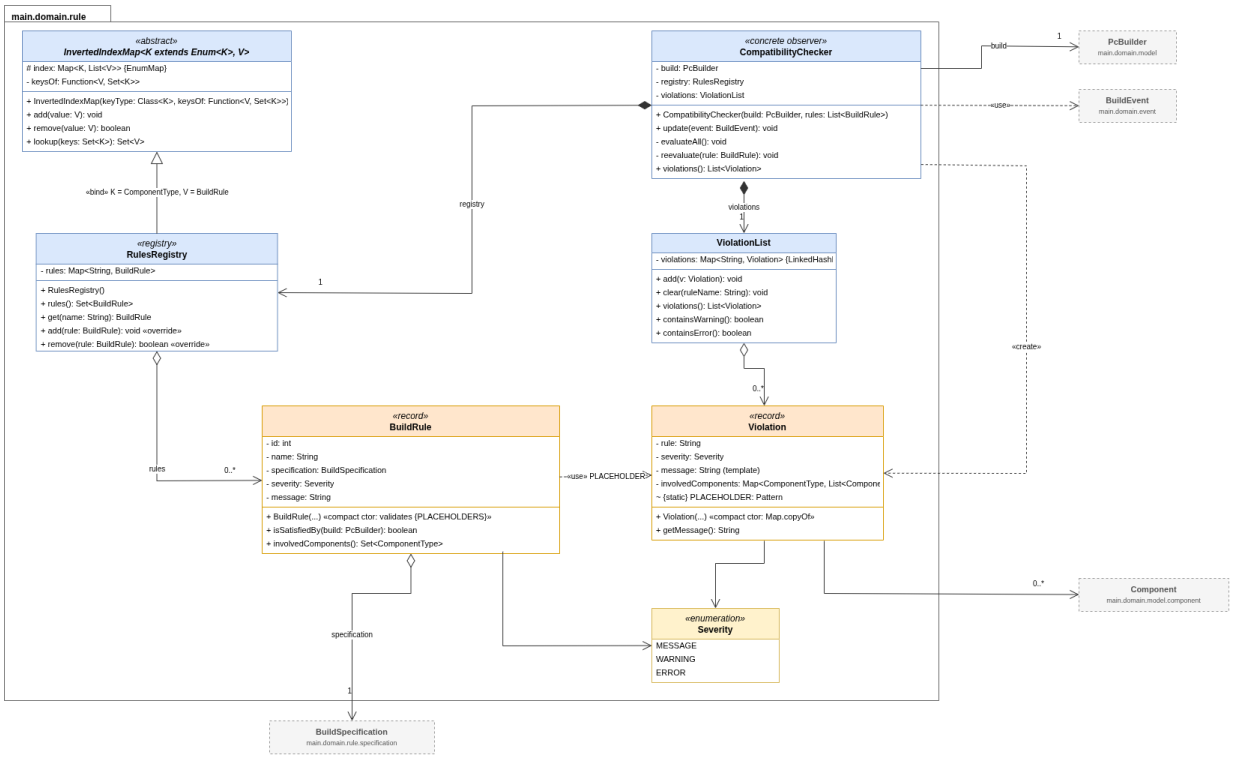


Figura 14: Enter Caption

Il vero cuore di questa applicazione è il sistema di controllo per verificare le compatibilità tra i vari componenti. Ogni componente ha delle specifiche relative alle proprie dimensioni o tecnologie utilizzate (ad esempio le grandezze dei case e quindi cosa può effettivamente contenere o se il socket della cpu corrisponde con quello della scheda madre). Bisogna anche tenere conto se il numero di componenti inseriti è corretto o se manca un componente essenziale per poter utilizzare il pc.

```java
//CompatibilityChecker.java
package main.domain.rule;

import main.domain.event.BuildEvent;
import main.domain.event.BuildEventType;
import main.domain.model.PcBuilder;
import main.domain.model.attribute.ComponentType;
import main.domain.model.component.Component;
import main.pattern.observer.Observer;

import java.util.*;

public class CompatibilityChecker implements Observer<
    BuildEvent> {
```

```
14      private final PcBuilder build;
15      private final RulesRegistry registry = new RulesRegistry();
16      private final ViolationList violations = new ViolationList
           ();
17
18      public CompatibilityChecker(PcBuilder build, List<BuildRule
           > rules) {
19          this.build = build;
20          build.attach(this);
21          // This lambda is the equivalent of r -> registry.add(r
               ), it was suggested by the IDE
22          rules.forEach(registry::add);
23          evaluateAll();
24      }
25
26      @Override
27      public void update(BuildEvent event) {
28          if (event.type() == BuildEventType.RESTORED)
29              evaluateAll();
30          else {
31              for (BuildRule rule : registry.lookup(Set.of(event.
                   componentType()))) {
32                  reevaluate(rule);
33              }
34          }
35      }
36
37      private void evaluateAll() {
38          for (BuildRule rule : registry.rules())
39              reevaluate(rule);
40      }
41
42      private void reevaluate(BuildRule rule) {
43          if (!rule.isSatisfiedBy(build)) {
44              Map<ComponentType, List<Component>> involved = new
                   LinkedHashMap<>();
45              for (ComponentType type : rule.involvedComponents()
                   ) {
46                  involved.put(type, List.copyOf(build.
                       getComponents(type)));
47              }
48              violations.add(new Violation(rule.name(), rule.
```

```
                    severity(), rule.message(), involved));
49              } else
50                  violations.clear(rule.name());
51          }
52
53          public List<Violation> violations() {
54              return violations.violations();
55          }
56      }
```

Listing 3: A simple Java class

Il motore principale con il quale vengono create le regole di compatibiltà si basa sul design pattern *Specification*: questo permette di definire business rules, con esito di validazione booleano, che possono essere combinate tra loro tramite operatori logici.

3.2 Application

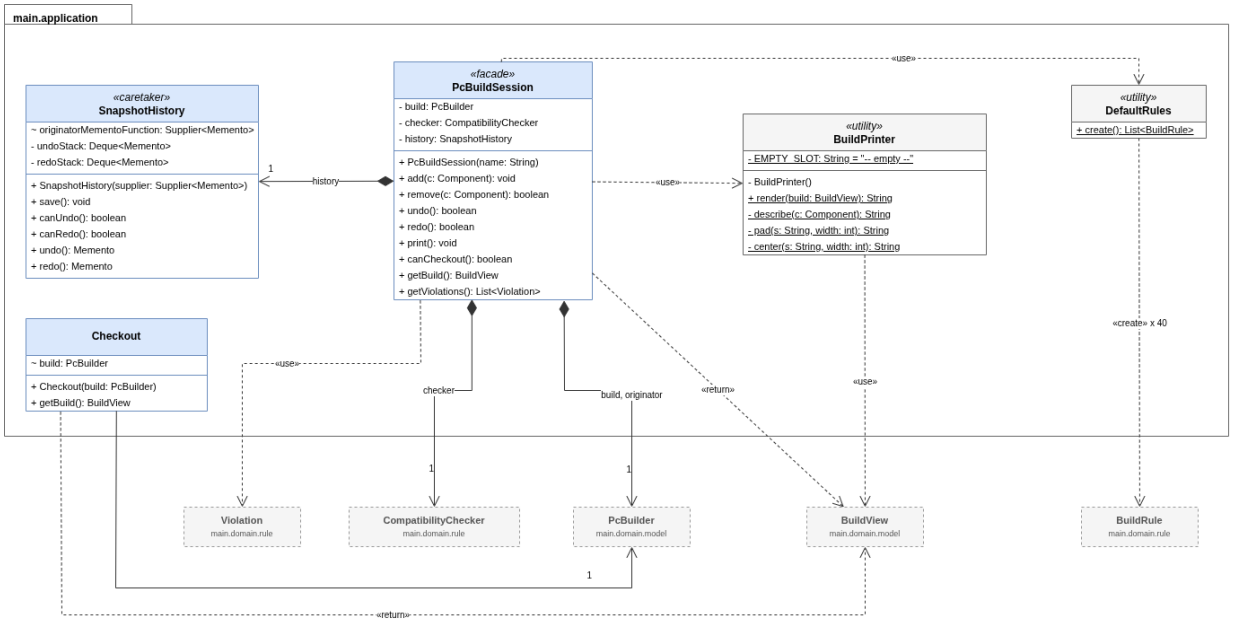


Figura 15: Enter Caption

Il pacchetto `application` contiene

3.3 ORM

TODO

4 Design Pattern e Strutture Dati

4.1 Specification

Specification è un design pattern definito da Eric Ewans in Domain Driven Design (da aggiungere ref).

- **Problema**: "You need to check that only suitable objects are used for a certain role", si parla quindi di responsabilità di validazione.
- **Soluzione**: "Create a specification that is able to tell if a candidate object matches some criteria. The specification has a method isSatisfiedBy(anObject): Boolean that returns "true" if all criteria are met by anObject".

Tra le varie implementazioni proposte, quella scelta è Composite Specification, che permette la massima flessibiltà e la possibilità di combinare le specifiche.

La struttura composita permette quindi di poter usare le operazioni di congiunzione logiche and e or) che compongono la specifica ad albero, con l'implementazione della versione composite di Specification si crea infatti un *interpreter* per la lingua della specifiche.

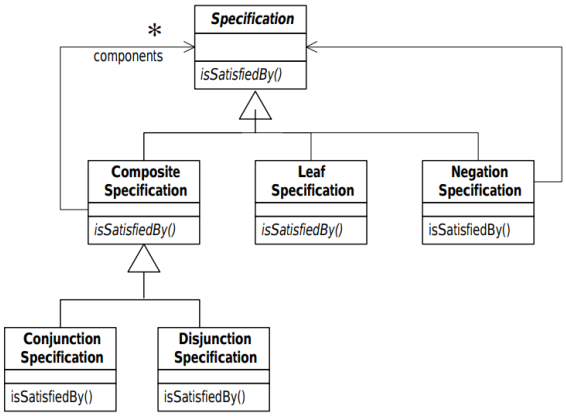


Figura 16: Enter Caption

$$SpecificationA \ \wedge \ (\neg SpecificationB \ \vee \ SpecificationC)$$

```
SpecificationTot = SpecificationA.and(SpecificationB.not().or(SpecificationC))
```
(1)

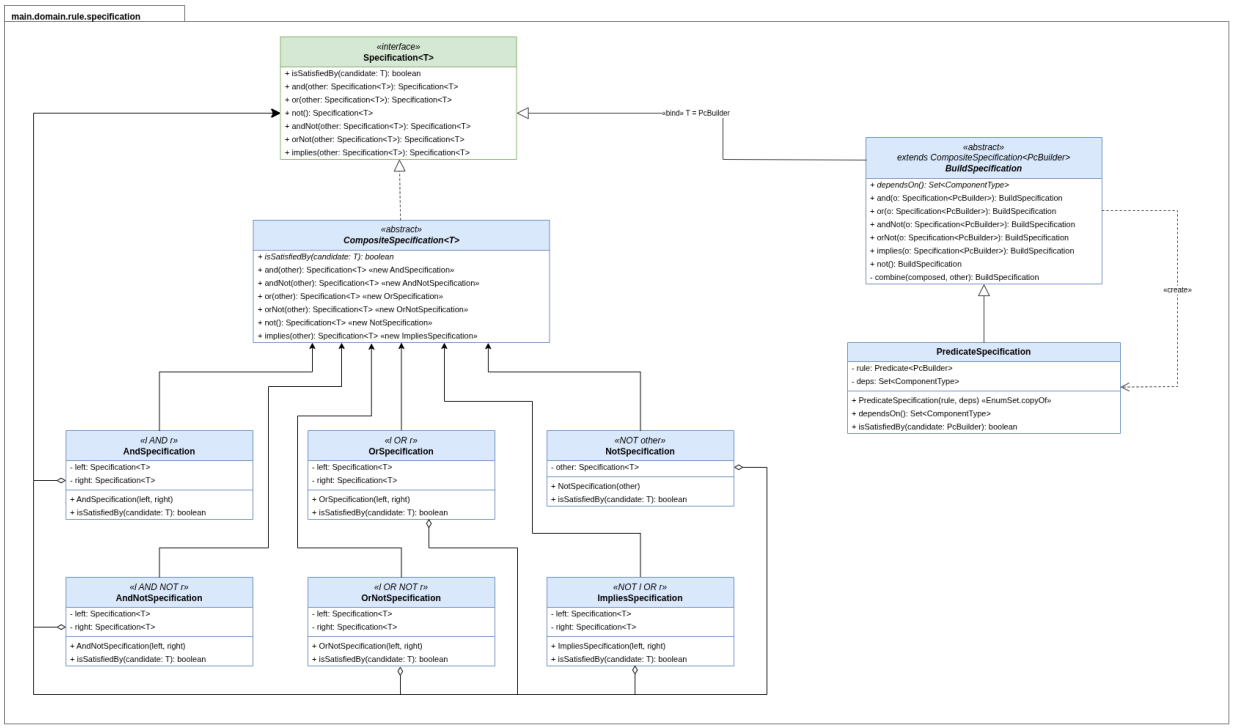


Figura 17: Enter Caption

```java
//Specification.java
package main.pattern.specification;

public interface Specification<T> {
    boolean isSatisfiedBy(T candidate);

    Specification<T> and(Specification<T> other);
    Specification<T> or(Specification<T> other);
    Specification<T> not();
    Specification<T> andNot(Specification<T> other);
    Specification<T> orNot(Specification<T> other);

    Specification<T> implies(Specification<T> other);

}
```

Listing 4: A simple Java class

```java
//CompositeSpecification.java
package main.pattern.specification;

public abstract class CompositeSpecification <T> implements
    Specification<T>{
```

```
5       @Override public abstract boolean isSatisfiedBy(T candidate
            );
6       @Override public Specification<T> and(Specification<T>
            other)    { return new AndSpecification<>(this, other);
            }
7       @Override public Specification<T> andNot(Specification<T>
            other) { return new AndNotSpecification<>(this, other);
            }
8       @Override public Specification<T> or(Specification<T> other
            )     { return new OrSpecification<>(this, other); }
9       @Override public Specification<T> orNot(Specification<T>
            other)  { return new OrNotSpecification<>(this, other);
            }
10      @Override public Specification<T> not()
                                    { return new NotSpecification
            <>(this); }
11      @Override public Specification<T> implies(Specification<T>
            other) { return new ImpliesSpecification<>(this, other);
             }
12  }
```

Listing 5: A simple Java class

4.1.1 Logica

4.2 Observer

4.3 Memento

4.4 Facade

4.5 Altri Design Pattern

4.6 Strutture Dati Utilizzate